

Information Security

Assignment – DVWA

Note: This is a weighted assignment. You are warned not to cheat because any plagiarism case will be treated equally.

Damn Vulnerable Web Application (DVWA) is a deliberately insecure web application designed for security professionals and students to practice common web vulnerabilities in a safe and legal environment.

In this assignment you will be setting up DVWA on your local machine and performing real world web application attacks. The goal is not just to understand what these vulnerabilities are, but to actually exploit them hands-on and understand how attackers think.

Tools Required:

1. XAMPP

- **Purpose:** Runs Apache web server, MySQL database and PHP locally on your machine.
- **Download:** <https://www.apachefriends.org/download.html>

2. DVWA (Damn Vulnerable Web Application)

- **Purpose:** The vulnerable web application we will be attacking.
- **Download:** <https://github.com/digininja/DVWA/archive/master.zip>
- **GitHub:** <https://github.com/digininja/DVWA>

3. Burp Suite Community Edition

- **Purpose:** Used for intercepting HTTP requests and performing Brute Force attacks.
- **Download:** <https://portswigger.net/burp/communitydownload>

4. FoxyProxy Extension

- **Purpose:** Browser extension to route traffic through Burp Suite easily.
- **Chrome:** <https://chromewebstore.google.com/detail/foxyproxy/gcknhkkoolaabfmlnjonogaaifnjlfnp>
- **Firefox:** <https://addons.mozilla.org/en-US/firefox/addon/foxyproxy-standard/>

PART A: (10 marks)

Instructions: From the following 5 vulnerabilities, choose any 3 and perform their exploitation on DVWA. Set security level to Low. For each vulnerability document every step with screenshots and brief explanation.

1. SQL Injection

- Navigate to SQL Injection lab and test the input field for vulnerability. What error do you get and what does it mean?
- Extract all users from the database using a payload that makes the condition always true.
- Extract all usernames and password hashes from the database using a UNION SELECT statement. Screenshot all results.
- What type of hashing is used on the passwords? Were you able to identify any real passwords from the hashes?

2. CSRF

- Navigate to CSRF lab and observe the Change Password form. What does this form do normally?
- Instead of using the form, craft a URL that directly changes the admin password and paste it in the browser. Screenshot the result.
- Create a fake HTML page that secretly sends a password change request when visited. What does the victim see vs what actually happens?
- Use Test Credentials to verify the password was changed. Screenshot the confirmation.

3. File Inclusion

- Navigate to File Inclusion lab in DVWA. What does this page normally do and how does it load files?
- Observe the URL carefully. What parameter is being used to load files? Try changing it to include a different local file from the server.
- Use the following path in the URL to read a sensitive file from the server
Command structure: `?page=../../hackable/flags/fi.php`
- Screenshot what you find.
- Try to include a file from outside the web directory. What information were you able to access and what does this tell you about the danger of file inclusion vulnerabilities?

4. XSS Reflected

- Navigate to XSS Reflected lab and type your name. What does the page do with your input?
- Instead of your name inject a basic JavaScript alert. Screenshot the popup that appears.
- Inject a script that displays your session cookie. Screenshot the cookie value.
- Explain in 2-3 lines what can an attacker do with a stolen session cookie?

5. XSS Stored

- Navigate to XSS Stored lab and post a normal message. What happens to the message?
- Post a JavaScript alert as your message. Screenshot the popup and explain why this is more dangerous than Reflected XSS.
- Refresh the page. Does the script fire again? Screenshot and explain what this means.
- How does Stored XSS affect every user who visits the page?

PART B: (10 Marks)

Instructions: Both of the following attacks are mandatory. Set security level to Low unless stated otherwise. Document every step with screenshots and brief explanation.

1. Brute Force

- Set up Burp Suite and FoxyProxy to intercept browser traffic.
- Intercept the login request in Brute Force lab. Identify all parameters being sent. Screenshot the raw request.
- Send to Intruder and set payload position on the **password field only**. Screenshot the configuration.
- Go to Payloads tab and load the following common passwords as your wordlist
- **Wordlist (passwords):** 123456, admin, helloWorld, abc123, charley
- Start the attack. Identify the successful credentials from results. What made this response different from failed attempts?
- Now switch security to **Medium** and try again. Did anything change? Were you still able to crack it?

2. File Upload

- Navigate to File Upload lab and upload a PHP web shell. Screenshot the success message.
- Execute **whoami** and **dir** through the shell. Screenshot each result.
- Now switch to **Medium** security and try uploading the same shell. What happens and why?
- Bypass the Medium security filter and successfully upload the shell again. Screenshot the bypass method you used.

What You Have to Submit:

- A complete PDF report documenting all steps with screenshots and explanations for every attack performed.
- Submit any code files created during the assignment. For example: your PHP web shell or any scripts used during exploitation.
- All files must be placed inside a single ZIP folder and submitted together.

Naming Conventions:

- **Failure to follow the naming convention will result in ZERO marks.**
- The table below shows the naming conventions you need to use.

File	Name Format
PDF Report	BCSF24 .pdf
Code File	BCSF24 .(file extension)
Zip Folder	BCSF24 .zip

Good luck!